

Programação de Sistemas

Programação com *structs* em C

Marcelo de Gomensoro Malheiros



Recapitulando...

» Criação de matrizes com base em elementos



- Impressão usando um laço encaixado:

- `int m[2][3] = { {4,5,6}, {7,8,9} };`
- `int m_lin = 2;`
- `int m_col = 3;`
- `for (int i = 0; i < m_lin; i++) {`
- `for (int j = 0; j < m_col; j++) {`
- `printf("%i ", m[i][j]);`
- `}`
- `printf("\n");`
- `}`

m	0	1	2
0	4	5	6
1	7	8	9

Recapitulando...

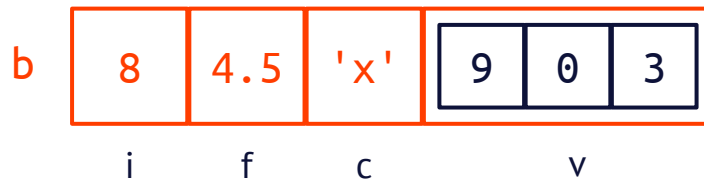
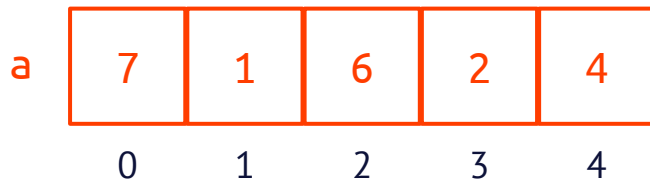


» Limitações de vetores e matrizes em C

- Operações de listas de Python que não podem em C:
 - teste de pertinência → é preciso percorrer e comparar
 - obter tamanho → é preciso guardar em uma variável
 - adicionar novo elemento → não é possível: vetores de C têm tamanho fixo
 - remover um elemento → não é possível: vetores de C têm tamanho fixo

Structs em C

- **Vetores** em C são variáveis compostas
 - Agrupam **elementos**, que são variáveis de um **mesmo tipo**
 - O acesso a cada variável se dá por um **índice** inteiro
- **Structs** em C também são variáveis compostas
 - Agrupam **membros**, que são variáveis que podem ter **tipos diferentes**
 - O acesso a cada variável se dá por um **nome** distinto

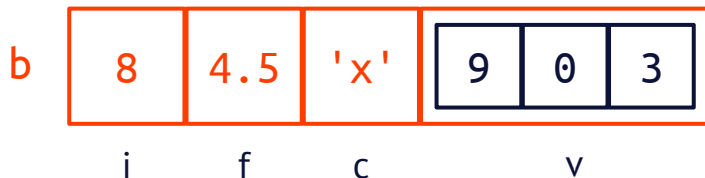


- Definição do tipo **B**:

- `typedef struct {`
- `int i;`
- `float f;`
- `char c;`
- `int v[3];`
- `} B;`

- Declaração e inicialização:

- Aqui criamos a variável **b** do tipo **B**
- Note que os valores precisam estar na ordem certa!
- `B b = {8, 4.5, 'x', {9, 0, 3}};`



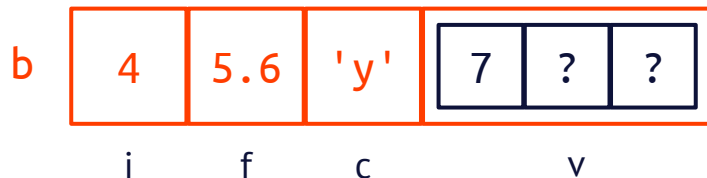
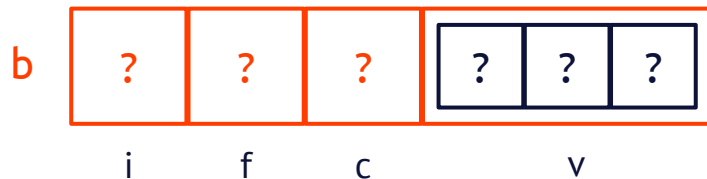
- Uma *struct* não inicializada também pode ser criada

- Neste caso todos os membros ficam com valores indefinidos

- **B** `b;`

- O acesso se dá com `.`

- `b.i = 4;`
- `b.f = 5.6;`
- `b.c = 'y';`
- `b.v[0] = 7;`



- Uma *struct* é um novo **tipo**, definido pelo programador
 - `typedef struct {`
 - `int x;`
 - `int y;`
 - `} Ponto;`
 - `typedef struct {`
 - `char nome[20];`
 - `int idade;`
 - `} Pessoa;`
- *Structs* são manipuladas como uma variável normal
 - `Ponto p1 = {100, 220};`
 - `Ponto p2 = p1; // cópia`
 - `Ponto p3;`
 - `p3.x = 350;`
 - `p3.y = p2.y;`
 - `Pessoa z = {"Ziraldo", 88};`
 - `printf("%i\n", z.idade);`

- *Structs* são definidas fora da função **main**

- Também se convencionou usar apenas a **inicial maiúscula**, para distinguir o nome de um tipo do nome de uma variável ou função

- Exemplo:

```
typedef struct {  
    char nome[20];  
    int idade;  
} Pessoa;  
  
int main(void) {  
    Pessoa p = {"Ana", 24};  
    printf("%s\n", p.nome);  
    return 0;  
}
```

- *Structs* também podem ser passadas por parâmetro

- Neste caso, uma **cópia** é feita de todos seus membros
- O mesmo **tipo** precisa ser usado na declaração da função
- `typedef struct {`
- `float i;`
- `float j;`
- `} Vetor;`

- Continuação:

```
○ float norma(Vetor v) {  
○     float n;  
○     n = v.i*v.i + v.j*v.j;  
○     return sqrt(n);  
○ }  
○ int main(void) {  
○     Vetor a = {1.0, -1.0};  
○     float d = norma(a);  
○     printf("d=%f\n", d);  
○ }
```

- *Structs* também podem ser retornadas por uma função
 - Neste caso, uma **cópia** é feita de todos seus membros
 - O **tipo** de retorno da função deve ser compatível

- Exemplo:

```
○ Vetor girar_90(Vetor v) {  
○     Vetor r;  
○     r.i = - v.j;  
○     r.j = v.i;  
○     return r;  
○ }  
○ Vetor girar_180(Vetor v) {  
○     Vetor r = {-v.i, -v.j};  
○     return r;  
○ }
```

Funções úteis

- São chamados de **casts** em C:
 - `(int) 1.0` → `1`
 - `(float) 1` → `1.0`
- Não é necessário converter de `int` para `char`

- `#include <stdlib.h>`
- Semente: `srand(1)`
- Gerador de números aleatórios:
 - `rand()` → 1804289383
 - faixa de valores inteiros gerados: 0 até `RAND_MAX`
- Receitas de bolo:
 - inteiros de 0 até 99: `rand() % 100`
 - inteiros de 100 até 200: `100 + rand() % (200 - 100 + 1)`
 - reais no intervalo [0, 1]: `(float) rand() / RAND_MAX`

Exercícios

- Faça um programa que defina uma *struct* chamada **Horario**, contendo hora **h** e minuto **m**.
 - Crie um procedimento para imprimir um **Horario** passado como parâmetro.
 - Crie uma função que recebe um **Horario** e um número **min** de minutos como parâmetro, retornando um novo horário com aqueles minutos adicionados.
 - Crie uma função que recebe um **Horario**, um número **hor** de horas e um número **min** de minutos, retornando um novo horário com aqueles dados adicionados.

Perguntas?